

2026 年 4 月 28 日 Version x.x

---

# ハッカソン ～計画書と設計書～

---

チーム○○○

2xG1xxx : FFFF NNNN  
2xG1xxx : FFFF NNNN  
2xG1xxx : FFFF NNNN  
2xG1xxx : FFFF NNNN  
2xG1xxx : FFFF NNNN  
2xG1xxx : FFFF NNNN



# 目次

|       |                        |   |
|-------|------------------------|---|
| 第 1 章 | プロジェクトの計画書             | 1 |
| 1.1   | プロジェクトの概要              | 1 |
| 1.1.1 | プロジェクトの題目              | 1 |
| 1.1.2 | プロジェクトの目的              | 1 |
| 1.1.3 | 既存の技術とプロジェクトの独自性       | 1 |
| 1.2   | スコープ・マネジメント            | 2 |
| 1.2.1 | 成果物                    | 2 |
| 1.2.2 | プロジェクトの対象範囲と非対象範囲      | 2 |
| 1.2.3 | 機能分解構造 FBS             | 2 |
| 1.2.4 | 製品分解構造 PBS             | 2 |
| 1.2.5 | FBS と PBS の対応関係        | 2 |
| 1.3   | 作業計画                   | 2 |
| 1.3.1 | 作業分解構造 WBS と管理番号       | 2 |
| 1.3.2 | アローダイアグラムとクリティカルパス     | 2 |
| 1.3.3 | 役割分担                   | 2 |
| 1.3.4 | ガントチャート                | 2 |
| 1.4   | 検証と妥当性確認: V&V 計画       | 2 |
| 1.4.1 | プロジェクトの成果物に対する有効指標 MOE | 2 |
| 1.4.2 | 有効指標 MOE に対応する性能指標 MOP | 2 |
| 1.4.3 | システムテストで評価する性能指標 MOP   | 2 |
| 1.4.4 | 結合テストで評価する性能指標 MOP     | 2 |
| 1.4.5 | 単体テストで評価する性能指標 MOP     | 2 |
| 1.4.6 | プロジェクト中に監視する技術性能指標 TPM | 2 |
| 1.4.7 | プロジェクト中に監視する指標 TPM     | 2 |
| 1.5   | 資源・調達管理                | 2 |
| 1.6   | リスク管理                  | 2 |
| 第 2 章 | システムの基本設計              | 3 |

|       |                                   |    |
|-------|-----------------------------------|----|
| 2.1   | 機能一覧                              | 3  |
| 2.1.1 | 機能分解 (FBS)                        | 3  |
| 2.1.2 | 機能と性能指標 (MOP) の対応                 | 4  |
| 2.2   | システム構成図                           | 4  |
| 2.2.1 | 全体ブロック図                           | 5  |
| 2.2.2 | ノード役割分担                           | 5  |
| 2.2.3 | データフロー                            | 6  |
| 2.2.4 | 採用アーキテクチャ (EMA) の方針               | 6  |
| 2.3   | 操作インターフェース設計                      | 7  |
| 2.3.1 | ユーザー操作                            | 7  |
| 2.3.2 | LED 表示仕様                          | 8  |
| 2.3.3 | 通信プロトコル方針 (CTRL / BEAT / NOTE)    | 8  |
| 2.3.4 | 通信方式 (WiFi UDP + USB Serial の二系統) | 9  |
| 2.3.5 | パケロス・ジッタ対策                        | 10 |
| 2.4   | 状態遷移図                             | 10 |
| 2.4.1 | 指揮者ノード (node_01)                  | 11 |
| 2.4.2 | 楽器ノード (node_02~05)                | 11 |
| 第 3 章 | システムの詳細設計                         | 13 |
| 3.1   | 部品一覧                              | 13 |
| 3.2   | ハードウェア設計                          | 14 |
| 3.2.1 | 配線・ピン                             | 15 |
| 3.2.2 | ネットワーク構成                          | 15 |
| 3.2.3 | 物理配置                              | 15 |
| 3.3   | ソフトウェア設計                          | 16 |
| 3.3.1 | ファイル構成                            | 16 |
| 3.3.2 | オブジェクト設計 (クラス図)                   | 17 |
| 3.3.3 | 関数設計                              | 18 |
| 3.4   | 処理フローの設計                          | 23 |
| 3.4.1 | 共通: 3 フェーズ実行モデル                   | 23 |
| 3.4.2 | 指揮者ノード (node_01)                  | 24 |
| 3.4.3 | 楽器ノード (node_02-05)                | 27 |
| 3.5   | テストの設計                            | 29 |

|       |                      |    |
|-------|----------------------|----|
| 3.5.1 | 試験戦略（単体 → 結合 → システム） | 29 |
| 3.5.2 | 試験項目（TPM）            | 30 |
| 3.5.3 | 要求 → 検証の対応           | 31 |



# 第 1 章 プロジェクトの計画書

## 1.1 プロジェクトの概要

### 1.1.1 プロジェクトの題目

### 1.1.2 プロジェクトの目的

プロジェクトの目的と最終的なゴール，解決すべき課題を記述する．

### 1.1.3 既存の技術とプロジェクトの独自性

## 1.2 スコープ・マネジメント

### 1.2.1 成果物

### 1.2.2 プロジェクトの対象範囲と非対象範囲

### 1.2.3 機能分解構造 FBS

### 1.2.4 製品分解構造 PBS

### 1.2.5 FBS と PBS の対応関係

## 1.3 作業計画

### 1.3.1 作業分解構造 WBS と管理番号

### 1.3.2 アローダイアグラムとクリティカルパス

### 1.3.3 役割分担

### 1.3.4 ガントチャート

## 1.4 検証と妥当性確認: V&V 計画

### 1.4.1 プロジェクトの成果物に対する有効指標 MOE

### 1.4.2 有効指標 MOE に対応する性能指標 MOP

### 1.4.3 システムテストで評価する性能指標 MOP<sup>2</sup>

### 1.4.4 結合テストで評価する性能指標 MOP



## 第 2 章 システムの基本設計

本章では、5 台の Arduino UNO R4 WiFi で構成する「Arduino オーケストラ」のうち塩澤担当範囲（指揮者ノード node\_01 と楽器ノード node\_02～node\_05 のファームウェア）を対象に、システム全体の機能・構成・操作インターフェース・状態遷移を基本設計レベルでまとめる。楽器構成は **金管 3 + ドラム 1** とし、各楽器ノードは**専用の Mac (Processing 実行) と 1 対 1 で USB 接続**される。Mac 側の音色合成・スピーカ出力（梅澤担当）は本章のスコープ外で、データ受け渡し境界（NOTE パケット）のみを定義する。通信は **ノード間（指揮者 → 楽器） = WiFi UDP マルチキャスト**、**Arduino-Mac 間（楽器 → 各 Mac） = USB Serial (UART, Serial.write)** の二系統に分離した構成を採る。

ファームウェアの設計原則は、外部 OSS であり著者本人が公開している **Embedded-Module-Architecture (以下 EMA)<sup>1)</sup>** に全面準拠する。EMA の中核ルール（IModule インターフェース、3 フェーズループ、SystemData と ProjectConfig の集約）は §2.2 で位置づけを示し、第 3 章で具体化する。

### 2.1 機能一覧

ファームウェア全体を機能分解構造（FBS）で整理する。本書スコープは Arduino 系のみであり、楽器音色や発表準備などチーム全体側の要素は含めない。

#### 2.1.1 機能分解（FBS）

- F1. 指揮情報抽出 (node\_01)** F1.1 IMU 6 軸の取得, F1.2 信号前処理 (IIR LPF + ノルム), F1.3 拍検出 (振り下ろしピーク), F1.4 テンポ推定 (拍間隔 → BPM), F1.5 強弱推定 (振幅 → velocity, ストレッチ), F1.6 演奏状態管理.
- F2. 指揮情報配信 (node\_01)** F2.1 CTRL 送出 (BPM・velocity・state), F2.2 BEAT 送出 (拍トリガ), F2.3 WiFi STA 接続維持・再接続.

---

1) <https://github.com/takushio2525/Embedded-Module-Architecture>

- F3. 楽譜進行 (node\_02~05)** F3.1 CTRL/BEAT 受信, F3.2 楽譜データ保持 (C 配列・パート別), F3.3 BEAT 同期での音符進行, F3.4 NOTE イベント生成, F3.5 パート固有ロジック (partId・startBeatNo).
- F4. 発音情報配信 (node\_02~05 → PC)** F4.1 NOTE パケット生成, F4.2 Processing への USB Serial 送信 (Serial.write(), CDC 1:1 接続).
- F5. 共通基盤 (全ノード)** F5.1 IModule 登録・3 フェーズループ, F5.2 ModuleTimer による周期実行, F5.3 SystemData/ProjectConfig 集約, F5.4 起動シーケンス・診断 LED, F5.5 init 失敗リトライ・パケロス時のフォールバック.

ストレッチ機能 (F1.5 強弱推定) は必達機能の実装完了後に着手する. 詳細設計ではインターフェースだけ用意し, 初期実装ではスタブで通す構成にする.

## 2.1.2 機能と性能指標 (MOP) の対応

各機能に対する性能指標を表 2.1 に示す. 目標値は原案段階のたたき台であり, 実機計測後に第 3.5 節で再調整する.

表 2.1: 機能と性能指標の対応

| ID    | 指標                           | 目標値                  |
|-------|------------------------------|----------------------|
| MOP-1 | 楽器間 同期誤差 (4 ノードの BEAT 受信時刻差) | $\leq 30 \text{ ms}$ |
| MOP-2 | 指揮 → 楽器 通信遅延 (送信～受信)         | $\leq 10 \text{ ms}$ |
| MOP-3 | 拍検出 正解率 (定速指揮時)              | $\geq 90 \%$         |
| MOP-4 | テンポ追従 遅延 (BPM 階段変化への追従)      | $\leq 2 \text{ 拍}$   |
| MOP-5 | パケロス耐性 (聴感破綻なし)              | 5 % パケロス             |
| MOP-6 | 入力フェーズ CPU 時間 (1 周期)         | $\leq 2 \text{ ms}$  |
| MOP-7 | 起動時間 (電源投入 → 演奏可能)           | $\leq 5 \text{ s}$   |
| MOP-8 | 強弱制御 分解能 (ストレッチ)             | 3 段階以上 (p/mf/f)      |

## 2.2 システム構成図

### 2.2.1 全体ブロック図

5 ノードと WiFi AP・Mac × 4 の関係を図 2.1 に示す。指揮者 node\_01 はローカル WiFi AP に対して CTRL/BEAT を **UDP マルチキャスト** で送信し、楽器 node\_02～05 はそれぞれグループに join してこれを受信する。各楽器は楽譜を進行させながら NOTE を **USB Serial (Serial.write)** で **専用 Mac (1 対 1 接続)** へ送信する。Arduino-Mac 間は WiFi を介さず、給電と Serial 通信を兼ねた USB 接続で直結する。

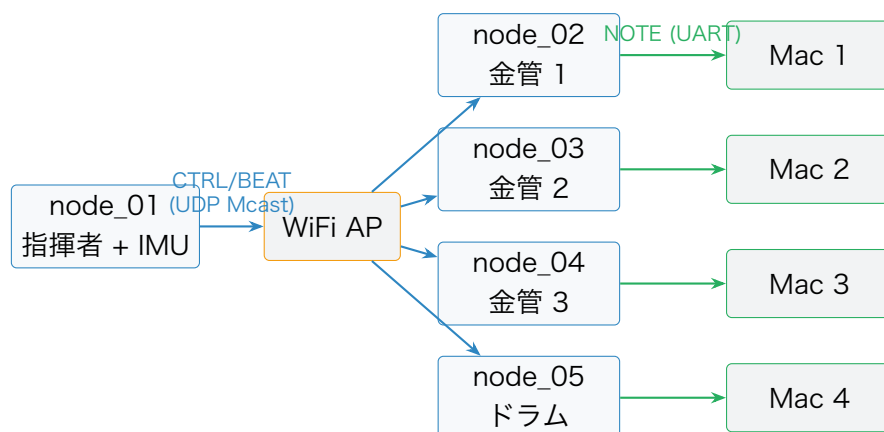


図 2.1 全体ブロック図（指揮者 1 + 楽器 4 + Mac 4, 楽器 → Mac は 1 対 1 USB Serial）

### 2.2.2 ノード役割分担

各ノードの責務とコード担当者を表 2.2 に示す。node\_02～05 の Arduino コードは **共通ソース + パート設定差し替え**（ProjectConfig と楽譜データのみ差分）で構成し、4 台分の別コードは書き分けない (§3.4.3)。

表 2.2: ノード役割分担

| ノード     | 物理配置          | 主な責務                                                   | コード担当   |
|---------|---------------|--------------------------------------------------------|---------|
| node_01 | 指揮者の手・指<br>揮棒 | IMU 読み取り → 拍・テンポ・強弱<br>推定 → CTRL/BEAT 送出               | 塩澤      |
| node_02 | 金管 1          | BEAT 同期で楽譜進行 → NOTE 送<br>出 (金管 1 パート)                  | 塩澤      |
| node_03 | 金管 2          | BEAT 同期で楽譜進行 → NOTE 送<br>出 (金管 2 パート)                  | 塩澤      |
| node_04 | 金管 3          | BEAT 同期で楽譜進行 → NOTE 送<br>出 (金管 3 パート)                  | 塩澤      |
| node_05 | ドラム           | BEAT 同期で楽譜進行 → NOTE 送<br>出 (ドラム)                       | 塩澤      |
| Mac × 4 | 楽器ごとに 1 台     | UART で NOTE を受信 → 倍音・<br>ADSR 調整 → 波形生成 → 内蔵ス<br>ピーカ出力 | 梅澤 (参考) |

### 2.2.3 データフロー

演奏中の 1 拍ぶんの情報の流れを擬似シーケンスで示す.

#### 1 拍ぶんのデータフロー (演奏中)

- (1) 内蔵 IMU → node\_01: 加速度・角速度サンプル (200 Hz).
- (2) node\_01 内: applyPattern() がフィルタ・拍検出・テンポ推定を実行.
- (3) 振り下ろしを検出: node\_01 → AP → node\_02~05 に **BEAT** (beat\_no, timestamp\_ms) をブロードキャスト.
- (4) node\_02~05 内: 楽譜ポインタ前進 → NoteOn フラグ設定.
- (5) node\_02~05 → 各 Mac (1 対 1) : **NOTE** (note\_number, velocity, duration\_ms) を USB Serial (Serial.write) で送信.
- (6) 並行して node\_01 は 20 Hz で **CTRL** (BPM・velocity・state) をブロードキャストし続ける.

## 2.2.4 採用アーキテクチャ (EMA) の方針

全 5 ノードのファームウェアは EMA に準拠する。EMA の中核ルールを本ハッカソンに当てはめた要旨を以下に示す。バイト単位のパケット定義や具体コードは第 3 章で示す。

- ・ **IModule インターフェース**: 全ハードウェアモジュールが `init()` / `updateInput()` / `updateOutput()` / `deinit()` の 4 メソッドを実装する。
- ・ **3 フェーズ実行モデル**: `loop()` は「入力 → ロジック `applyPattern()` → 出力」の順で必ず実行する。
- ・ **SystemData 集約**: モジュール間共有状態を一元化する構造体。モジュール同士の直接呼び出しは禁止。
- ・ **ProjectConfig 集約**: 各モジュールの設定を `{MODULE}_CONFIG` インスタンスとして 1 ヘッダに集約。
- ・ **判断ロジックの位置づけ**: 拍検出・テンポ推定・楽譜進行のような「ハードウェアに触れない判断ロジック」は `IModule` の派生にせず、`applyPattern()` 関数内を書く。

## 2.3 操作インターフェース設計

本節では「操作インターフェース」を、**ユーザーが触れる物理 IF** (指揮棒・LED) と **ノード間 / PC 間の通信プロトコル IF** (CTRL/BEAT/NOTE) の二層に分けて定義する。通信は **Arduino-Arduino 間 = WiFi UDP**, **Arduino-PC 間 = USB Serial** の二系統に分け、それぞれ役割と性質が異なる。バイト単位のパケットレイアウトは §3.3.3 で具体化する。

### 2.3.1 ユーザー操作

- ・ **指揮棒 (指揮者)**: 内蔵 IMU を持つ `node_01` を指揮棒として手に持ち、通常の指揮動作 (4 拍子・3 拍子の振り下ろし) で操作する。特別なボタン等はない。

- ・ **起動順**: 楽器ノード (node\_02~05) を先に起動して WaitStart 状態にし, その後に指揮者ノード (node\_01) を起動する. 指揮者の Conducting 入りで BEAT が流れ始め, 楽器が Playing に遷移する.
- ・ **Calibrating**: 指揮者ノードは起動後, 最初の 2 秒間に静止姿勢を学習して重力オフセットを記録する. 使用者は単に「起動後 2 秒間, 腕を自然に下ろした状態で待つ」だけでよい.

## 2.3.2 LED 表示仕様

各ノードは内蔵 LED (LED\_BUILTIN) の点滅パターンで状態を提示する (表 2.3). 点滅周期は StatusLedConfig.blinkIntervalMs で調整可能.

表 2.3: LED 表示パターン

| ノード        | 状態          | LED パターン    | 意味                    |
|------------|-------------|-------------|-----------------------|
| node_01    | Idle        | 低速点滅 (1 Hz) | 起動直後・WiFi 未接続         |
| node_01    | Calibrating | 中速点滅 (2 Hz) | 重力オフセット学習中 (2 秒)      |
| node_01    | Conducting  | 点灯          | 通常演奏中                 |
| node_01    | Fallback    | 高速点滅 (5 Hz) | IMU エラー or WiFi 切断    |
| node_02-05 | Idle        | 低速点滅 (1 Hz) | 起動直後・WiFi 未接続         |
| node_02-05 | WaitStart   | 中速点滅 (2 Hz) | WiFi 接続済, 曲開始 BEAT 待ち |
| node_02-05 | Playing     | 点灯          | 通常演奏中                 |
| node_02-05 | SelfRun     | 高速点滅 (5 Hz) | BEAT 取りこぼし検出, 自走中     |

## 2.3.3 通信プロトコル方針 (CTRL / BEAT / NOTE)

3 種類のメッセージを使い分ける (表 2.4).

表 2.4: 3 種類のメッセージの設計方針

| 種別   | 送信元 → 宛先                              | 送信契機                  | 性質        | 役割                           |
|------|---------------------------------------|-----------------------|-----------|------------------------------|
| CTRL | node_01<br>node_02-05<br>Mcast)       | → 定期 (20 Hz)<br>(UDP  | 冪等        | BPM・velocity・<br>state を継続同期 |
| BEAT | node_01<br>node_02-05<br>Mcast)       | → 拍検出時 (不<br>(UDP 定期) | 即時性重<br>視 | 楽譜ポイント前進<br>トリガ              |
| NOTE | node_02-05 →<br>Mac (USB Serial, 1:1) | 各 楽譜イベント<br>発火時       | 順序保証      | 自パート Mac へ<br>の発音依頼          |

#### 設計原則:

- ・ **CTRL は冪等**: 取りこぼしてもすぐ次が来るため, パケロス耐性は通信方針側で確保される.
- ・ **BEAT は即時**: 1 拍 1 発が原則. 確実性を上げる場合は同一 beat\_no を 2~3 連送し, 受信側で beat\_no 重複排除する (冪等処理).
- ・ **楽譜側内挿**: BEAT が一定時間 (既定 1500 ms) 来なければ楽器が自走 (SelfRun) して直前 BPM から拍を内挿する.
- ・ **NOTE は単一経路**: 楽器 → 自パート Mac で 1 系統. 楽譜に従って必ず送出する. Arduino-Mac 間は 1 対 1 USB Serial 直結のため, パケロスやネットワーク輻輳の心配なく確定到達するのが利点.

### 2.3.4 通信方式 (WiFi UDP + USB Serial の二系統)

通信仕様を表 2.5 に示す. 運用時に変わる値 (SSID / パス) は OrcNetConfig のリテラルで一元管理する.

表 2.5: 通信仕様

| 項目                 | 設計                                                                                                                              |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------|
| ノード間 (CTR-L/BEAT)  | WiFi UDP, node_01 → node_02-05 へ <b>UDP/5001 マルチキャスト</b> (仮 239.0.0.1). 楽器側は <code>WiFiUDP.beginMulticast()</code> でグループに join  |
| WiFi モード           | 全 Arduino ノード STA, 外部 AP (スマホテザリング or 専用ルータ) へ接続                                                                                |
| SSID / パス          | 仮 OrchestraAP / orchestra2026, 本番運用で差替え                                                                                         |
| IP 割り当て            | DHCP (初期段階). 固定 IP が必要になれば <code>OrcNetConfig</code> で切替可能                                                                      |
| Arduino-Mac (NOTE) | 間 <b>USB Serial (CDC, <code>Serial.write</code>)</b> . <b>各楽器ノードと専用 Mac が 1 対 1</b> で USB 接続. Mac 側 Processing は対応する 1 ポートだけを開く |
| ボーレート              | 115200 bps (UNO R4 USB CDC は実質高速だが値は揃える). <code>NoteSenderConfig.baudRate</code> で集約                                            |
| フレーミング             | 全パケットの先頭 12 B 共通ヘッダの <code>magic = 0x4F52</code> がフレーム同期マークを兼ねる (§3.3.3)                                                        |
| エンディアン             | リトルエンディアン (UNO R4 ARM Cortex-M4 ネイティブ)                                                                                          |

### 2.3.5 パケロス・ジッタ対策

- ・ **CTRL 取りこぼし**: 冪等かつ 20 Hz 定期送信なので設計上問題にならない.
- ・ **BEAT 取りこぼし**: 同一 BEAT を 2~3 連発 + 楽器側で `beat_no` 重複排除. さらに直前 BPM から予測時刻を内挿し, 一定時間 BEAT が来なかったら自走 (SelfRun).
- ・ **ジッタ**: BEAT 受信時刻で楽譜ポインタを前進させ, 内挿は最後の手段にとどめる.
- ・ **プロトコル不整合**: `magic / version` 不一致のパケットは破棄し, `StatusLedModule` が `data.orcNet` を読んで点滅パターンに反映する.



## 2.4 状態遷移図

各ノード種ごとの演奏状態を 4 状態の有限状態機械として定義する。状態は ConductorState (指揮者) と PerformerState (楽器) の enum class として SystemData の中に保持し、applyPattern() が遷移を判定する。

### 2.4.1 指揮者ノード (node\_01)

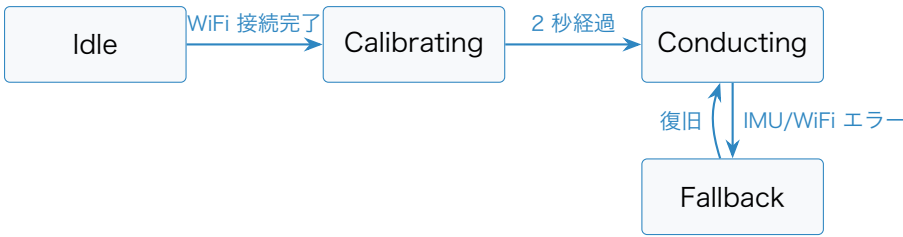


図 2.2 指揮者ノードの状態遷移

表 2.6: 指揮者ノードの状態定義

| 状態          | 意味               | 動作                                                       |
|-------------|------------------|----------------------------------------------------------|
| Idle        | 起動直後、WiFi 未接続    | ImuModule.updateInput は動くが拍検出は無効、OrcSenderModule は何も送らない |
| Calibrating | 初期静止姿勢の学習中 (2 秒) | applyPattern() が IMU 平均で重力オフセットを記録                       |
| Conducting  | 通常演奏中            | 全モジュール稼働。BEAT を拍検出時に送出、CTRL を 20 Hz 周期で送出                |
| Fallback    | エラー発生            | CTRL は state=Fallback で送り続ける。BEAT は停止、LED 高速点滅           |

### 2.4.2 楽器ノード (node\_02~05)



図 2.3 楽器ノードの状態遷移

表 2.7: 楽器ノードの状態定義

| 状態        | 意味                   | 動作                                       |
|-----------|----------------------|------------------------------------------|
| Idle      | 起動直後, WiFi 未接続       | 受信モジュールは動くが楽譜進行ロジックは無効                   |
| WaitStart | WiFi 接続済, 曲頭 BEAT 待ち | beat_no が startBeatNo に到達するまで待機          |
| Playing   | 通常演奏中                | BEAT 受信 → 楽譜進行 → NOTE 送出                 |
| SelfRun   | BEAT 取りこぼし検出, 自走中    | 直前 BPM から仮想 BEAT を内挿し, 楽譜進行を継続. LED 高速点滅 |

## 第 3 章 システムの詳細設計

本章では、第 2 章で示した基本設計を実装可能な水準まで具体化する。塩澤担当範囲（WBS タスク 121「共通層 API 詳細」、122「指揮者ノード詳細」、123「楽器ノード詳細」）の 3 領域を中心に、部品・ハードウェア配線・ソフトウェア構造・処理フロー・テストを順に記す。コード例は要点抽出にとどめ、フル実装は本書スコープ外とする。

### 3.1 部品一覧

製品分解構造（PBS）として、本ファームウェアが直接関わるハードウェア・ソフトウェア構成要素を表 3.1・表 3.2 に列挙する。PC 側 Processing 環境やスピーカは本書スコープ外であるが、NOTE パケットの受信境界として記述する。

表 3.1: ハードウェア部品一覧（PBS）

| 区分        | 要素                  | 数量 | 備考                                                 |
|-----------|---------------------|----|----------------------------------------------------|
| P1.1 指揮者  | Arduino UNO R4 WiFi | 1  | Renesas RA4M1 + ESP32-S3                           |
| P1.1      | 内蔵 IMU（LSM6DSOX 相当） | 1  | 加速度 $\pm 4g$ / 角速度 $\pm 2000$ dps                  |
| P1.1      | 内蔵 WiFi             | 1  | WiFiS3 で制御                                         |
| P1.1      | 状態 LED              | 1  | LED_BUILTIN 流用                                     |
| P1.2 楽器   | Arduino UNO R4 WiFi | 4  | node_02-05 同一 H/W. <b>金管 1 / 金管 2 / 金管 3 / ドラム</b> |
| P1.2      | 内蔵 WiFi             | 4  | 受信専用（指揮者からの UDP マルチキャスト）                           |
| P1.2      | 状態 LED              | 4  | LED_BUILTIN 流用                                     |
| P2 ネットワーク | ローカル WiFi AP        | 1  | スマホテザリング <sup>a</sup> or 専用ルータ                     |

| 区分         | 要素               | 数量 | 備考                                                                        |
|------------|------------------|----|---------------------------------------------------------------------------|
| P3 給電・通信   | USB Type-C ケーブル  | 5  | 楽器 4 本は <b>各 Mac へ 1 対 1 直結</b> (給電 + Serial 通信兼用). 指揮者ノードは USB ハブ経由で給電のみ |
| P4 境界 (参考) | Mac (Processing) | 4  | 本書スコープ外. 各楽器から USB Serial で NOTE を受信し, 倍音・ADSR 調整・波形生成を行う                 |
| P4 境界 (参考) | 内蔵スピーカ           | 4  | 本書スコープ外. Mac 1 台あたり 1 系統                                                  |

表 3.2: ソフトウェア技術スタック

| レイヤ      | 採用技術                                     | バージョン / 備考                     |
|----------|------------------------------------------|--------------------------------|
| MCU      | UNO R4 WiFi                              | Renesas RA4M1 + ESP32          |
| フレームワーク  | Arduino Framework                        | PlatformIO 経由                  |
| 言語       | C++ (C++11)                              | arduino-core 範囲                |
| ビルド      | PlatformIO                               | platform=renesas-ra, board=... |
| 外部ライブラリ  | Arduino_LSM6DSOX                         | IMU 読み取り                       |
| 外部ライブラリ  | WiFiS3                                   | 標準同梱の WiFi/UDP                 |
| 設計パターン   | EMA                                      | 全章前提 (ADR-0005)                |
| 共通層 (流用) | ModuleCore (IModule + ModuleTimer)       | EMA からそのまま流用                   |
| 自作層 (新規) | OrcProtocol / OrcNetModule               | §3.3.3 で定義                     |
| ネットワーク   | UDP over WiFi (ノード間) + USB Serial (PC 間) | 二系統に分離                         |

## 3.2 ハードウェア設計

### 3.2.1 配線・ピン

指揮者ノード (node\_01) は内蔵 IMU を使うため I2C を経由する。楽器ノードは内蔵 LED 以外に外部配線を要しない (表 3.3)。

表 3.3: 配線・ピンアサイン

| ノード        | 用途               | ピン                                          |
|------------|------------------|---------------------------------------------|
| node_01    | I2C SDA (内蔵 IMU) | D18 (I2C_SDA_PIN = 18)                      |
| node_01    | I2C SCL (内蔵 IMU) | D19 (I2C_SCL_PIN = 19)                      |
| node_01    | 状態 LED           | LED_BUILTIN                                 |
| node_01    | 給電               | USB Type-C (USB ハブ経由, Serial は使用しない)        |
| node_02-05 | 状態 LED           | LED_BUILTIN                                 |
| node_02-05 | 給電 + NOTE 送信     | USB Type-C (各楽器が専用 Mac へ 1 対 1 直結, Serial.) |

### 3.2.2 ネットワーク構成

通信は二系統に分離する。Arduino 同士は WiFi UDP (同一 LAN, AP 配下) で, Arduino と Mac は USB Serial 直結で通信する。node\_01 は CTRL/BEAT を UDP マルチキャスト (仮 239.0.0.1:5001) で送信し, node\_02-05 はそれぞれグループに join して受信する。NOTE は各楽器ノードが 専用 Mac のシリアルポートへ Serial.write で書き込む (楽器 ↔ Mac は 1 対 1)。本番運用時の SSID / パスは OrcNetConfig のリテラルで切替可能とする。Mac 側の Processing は自分に繋がっているシリアル 1 ポートだけを開いて受信する。

### 3.2.3 物理配置

演奏時は指揮者を中心に楽器 4 台を扇形に配置し, 各 Mac とそれぞれの内蔵スピーカは観客側からは見えない位置に並べる (楽器 → Mac は USB ケーブルで直

結). 指揮者ノードの給電は USB ハブで集約する. 配置に関する具体寸法・治具設計はチーム全体側 (梅澤担当) に委ねる.

## 3.3 ソフトウェア設計

### 3.3.1 ファイル構成

firmware/ 配下は **共通層**と**ノード別プロジェクト**の2段構造にする. EMA に準拠し, PlatformIO の lib\_extra\_dirs で共通層を全ノードから参照する.

#### firmware/ ディレクトリ構成 (抜粋)

```
firmware/
|-- common/
|   |-- lib/
|       |-- ModuleCore/                # コア層 (プロジェクト非依存)
|       |   |-- IModule.h
|       |   |-- ModuleTimer.h
|       |-- OrcProtocol/                # CTRL/BEAT/NOTE のパケット定義
|       |   |-- OrcProtocol.h
|       |   |-- OrcProtocol.cpp
|       |-- OrcNetModule/               # WiFi 接続維持 + UDP 送受信
|       |   |-- OrcNetModule.h
|       |   |-- OrcNetModule.cpp
|-- node_01/                            # 指揮者ノード
|   |-- platformio.ini
|   |-- include/
|       |-- SystemData.h                # {Module}Data を集約
|       |-- ProjectConfig.h             # {MODULE}_CONFIG + 共有バスピン
|   |-- src/
|       |-- main.cpp                    # 入出力配列・3 フェーズループ
|       |-- applyPattern.cpp             # フィルタ/拍検出/テンポ推定/状態遷移
|   |-- lib/
|       |-- ImuModule/                   # 入力: IMU
|       |-- OrcSenderModule/             # 出力: CTRL/BEAT 送信
|       |-- StatusLedModule/             # 出力: 状態 LED
|-- node_02/                            # 楽器ノード A
|   |-- platformio.ini
|   |-- include/
|       |-- SystemData.h
|       |-- ProjectConfig.h             # partId=0x02, PC IP, startBeatNo 等
|       |-- score_data.h                 # パート A の楽譜 (C 配列)
|   |-- src/
```

```

| | |-- main.cpp
| | |-- applyPattern.cpp          # 楽譜進行 + SelfRun 判定
| |-- lib/
|     |-- OrcReceiverModule/      # 入力: CTRL/BEAT 受信
|     |-- NoteSenderModule/      # 出力: NOTE 送信
|     |-- StatusLedModule/
|-- node_03/ (構造同じ。ProjectConfig と score_data.h が差分)
|-- node_04/
|-- node_05/

```

各ノードの platformio.ini には次の 2 行を必ず含める (EMA 必須要件)。

```

build_flags    = -I include          ; lib/ から include/SystemData.h を参照するために必須
lib_extra_dirs = ../common/lib       ; ModuleCore / OrcProtocol / OrcNetModule を共有

```

### 3.3.2 オブジェクト設計 (クラス図)

EMA の IModule 抽象基底をすべてのハードウェアモジュールが継承する。OrcNetModule は共通層に置き両ノード種で共有する (図 3.1)。

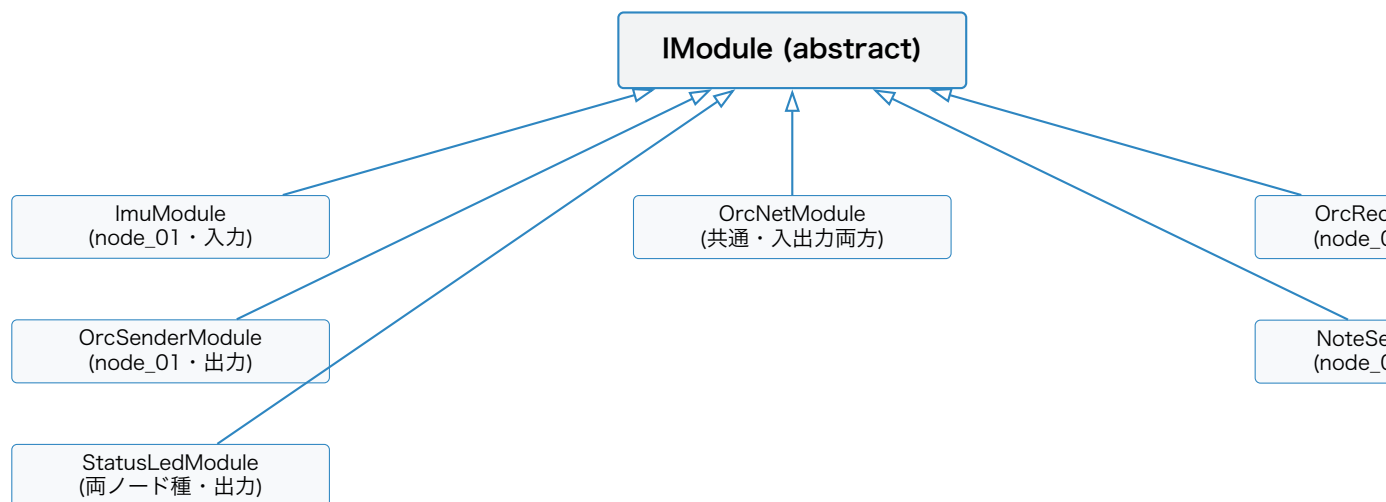


図 3.1 モジュールのクラス継承関係

表 3.4: モジュール一覧（入出力分類別）

| モジュール             | ノード                | 分類  | 責務                                                          |
|-------------------|--------------------|-----|-------------------------------------------------------------|
| ImuModule         | node_01            | 入力  | 内蔵 IMU 6 軸を読み data.imu に書く                                  |
| OrcSenderModule   | node_01            | 出力  | data.beat/data.tempo を読み CTRL/BEAT 送信キューに投入                 |
| OrcReceiverModule | node_02-05         | 入力  | data.orcNet.lastCtrl/Beat を読み data.receiver に整形             |
| NoteSenderModule  | node_02-05         | 出力  | data.noteOut.pendingOn/Off を見て NOTE を Serial.write で PC へ送信 |
| OrcNetModule      | node_01・node_02-05 | 入出力 | WiFi 接続維持, UDP 受信ポーリング（入力）と CTRL/BEAT 送信キューフラッシュ（出力）        |
| StatusLedModule   | 全ノード               | 出力  | 状態に応じた LED 点滅パターン表示                                         |

### 3.3.3 関数設計

#### 3.3.3.1 IModule インターフェース（流用）

##### IModule.h

```

1 struct SystemData; // 前方宣言（プロジェクト依存を避ける）
2
3 class IModule {
4 public:
5     bool enabled = true;
6     virtual bool init() = 0; // 純粋仮想
7     virtual void updateInput(SystemData& data) {} // デフォルト空
8     virtual void updateOutput(SystemData& data) {} // デフォルト空
9     virtual void deinit() {} // デフォルト空
10    virtual ~IModule() {}

```



```
11 };
```

init() の失敗時は setup() 側で MAX\_RETRY 回リトライし、それでも失敗した場合は enabled = false とする。ロジックフェーズで定期的に再 init() を試みて enabled = true に復帰させる (§3.4.2 参照)。

### 3.3.3.2 ModuleTimer (流用)

#### ModuleTimer.h

```
1 class ModuleTimer {
2 public:
3     void      setTime();                // 現在時刻を起点としてセット
4     uint32_t  getNowTime() const;       // setTime() からの経過時間 (ms)
5 private:
6     uint32_t  _startMs = 0;
7 };
```

サンプリング周期 (IMU 200 Hz → intervalMs=5), CTRL 送信周期 (20 Hz → 50 ms), LED 点滅, BEAT 不応期, SelfRun タイムアウト判定などに使用する。

### 3.3.3.3 通信パケット定義 (OrcProtocol)

3 種のパケットすべてに 12 バイトの共通ヘッダを付与する。データはリトルエンディアンでシリアライズする。CTRL/BEAT は WiFi UDP マルチキャスト, NOTE は USB Serial (バイナリ列を直接書込) で転送する。magic = 0x4F52 は WiFi UDP ではプロトコル識別子, USB Serial では **フレーム同期マーク**を兼ねる (Mac 側は magic を探して同期し, type でペイロード長を判定して残りを読む)。

### 共通ヘッダ (12 バイト)

| オフセット | フィールド        | 型        | 説明                      |
|-------|--------------|----------|-------------------------|
| 0     | magic        | uint16_t | 固定値 0x4F52 (ASCII "OR") |
| 2     | version      | uint8_t  | 初期値 0x01                |
| 3     | type         | uint8_t  | 1=CTRL, 2=BEAT, 3=NOTE  |
| 4     | seq          | uint32_t | 単調増加 (パケロス検出用)          |
| 8     | timestamp_ms | uint32_t | 送信側 millis()            |

### CTRL ペイロード (type

| オフセット | フィールド    | 型          | 説明                                              |
|-------|----------|------------|-------------------------------------------------|
| 12    | bpm_q8   | uint16_t   | BPM × 8 (例: 120.5 → 964)                        |
| 14    | velocity | uint8_t    | 0-127. ストレッチ未実装時は固定 64                          |
| 15    | state    | uint8_t    | 0=Idle, 1=Calibrating, 2=Conducting, 3=Fallback |
| 16    | 予約       | uint8_t[4] | ゼロ埋め                                            |

### BEAT ペイロード (type

| オフセット | フィールド    | 型          | 説明                       |
|-------|----------|------------|--------------------------|
| 12    | beat_no  | uint16_t   | 曲頭からの拍番号 (曲開始時に 0 リセット)  |
| 14    | phase_q8 | uint16_t   | 拍内位相 (0-255 で 1 拍). 通常 0 |
| 16    | 予約       | uint8_t[4] | ゼロ埋め                     |

#### NOTE ペイロード (type

| オフセット | フィールド       | 型          | 説明                        |
|-------|-------------|------------|---------------------------|
| 12    | part_id     | uint8_t    | 0x02-0x05 (= node_02-05)  |
| 13    | note_number | uint8_t    | MIDI ノート番号 (0-127, 60=C4) |
| 14    | velocity    | uint8_t    | 0-127                     |
| 15    | gate        | uint8_t    | 1=NoteOn, 0=NoteOff       |
| 16    | duration_ms | uint16_t   | 発音予定長さ (参考値)              |
| 18    | 予約          | uint8_t[2] | ゼロ埋め                      |

#### 3.3.3.4 OrcNetModule API (WiFi UDP 専用)

WiFi 接続維持と UDP 送受信ポーリングを集約する IModule 実装. CTRL/BEAT のみを担当し, NOTE は別モジュール (NoteSenderModule) が USB Serial で直送する. 入出力両方の責務を持つため, inputModules[] と outputModules[] の両方に登録する.

#### OrcNetModule.h (抜粋)

```
1 struct OrcNetConfig {
2     const char* ssid;
3     const char* pass;
4     IPAddress    multicastIp;           // 例: IPAddress(239, 0, 0, 1)
5     uint16_t     udpPort;               // CTRL/BEAT 共通ポート (5001)
6     uint32_t     reconnectIntervalMs = 2000;
7 };
8
9 struct OrcNetData {
10     bool         wifiConnected = false;
11     uint8_t      sendQueueDepth = 0;
12     bool         hasNewCtrl      = false;
13     bool         hasNewBeat     = false;
14     CtrlPacket   lastCtrl       = {};
15     BeatPacket   lastBeat       = {};
16 };
17
18 class OrcNetModule : public IModule {
19 public:
20     explicit OrcNetModule(const OrcNetConfig& config);
```

```

21     bool init() override;                                // WiFi.begin() + Udp.
                    beginMulticast()
22     void updateInput(SystemData& data) override;         // WiFi 状態 + 受信ポーリ
                    ング
23     void updateOutput(SystemData& data) override;        // CTRL/BEAT 送信キューフ
                    ラッシュ
24     void deinit() override;
25
26     // applyPattern() / 他モジュールから呼ぶ送信 API (CTRL/BEAT のみ)
27     bool enqueueCtrl(const CtrlPacket& p);              // multicastIp:udpPort へ送信
28     bool enqueueBeat(const BeatPacket& p);              // multicastIp:udpPort へ送信
29 };

```

**ルール:** enqueueXxx() はロジックフェーズで呼び、実際の Udp.send() は updateOutput() で一括処理する。tryRecvXxx() のような外向き受信 API は持たず、受信結果は OrcNetData 内のフラグ + 直近パケットバッファとして SystemData 経由で渡す（モジュール間直接呼び禁止のため）。

### 3.3.3.5 NoteSenderModule API (USB Serial 専用)

楽器ノード (node\_02-05) が NOTE をハードウェア Serial へ書き出すための IModule 実装。init() で Serial.begin(baudRate) を呼び、updateOutput() で data.noteOut.pendingOn/Off フラグを見て NotePacket を構築 → 20 バイトのバイナリを Serial.write() で一括送出する。

#### NoteSenderModule.h (抜粋)

```

1  struct NoteSenderConfig {
2      uint32_t baudRate = 115200;    // USB CDC ボーレート
3      uint8_t  partId;               // 0x02..0x05 (パケット内識別子。1:1 接続
                                     // のため Mac 側では参考値)
4  };
5
6  struct NoteSenderData {
7      uint32_t noteSeq    = 0;
8      uint32_t lastSentMs = 0;
9  };
10
11 class NoteSenderModule : public IModule {
12 public:
13     explicit NoteSenderModule(const NoteSenderConfig& config);

```

```

14     bool init() override;                                // Serial.begin(baudRate
    )
15     void updateOutput(SystemData& data) override;        // pendingOn/Off ->
        Serial.write
16 };

```

Mac 側 Processing は、自パートの楽器ノードに繋がっている 1 つの USB シリアルポートを開き、magic 0x4F52 を探してフレーム同期し、type=3 のとき NOTE ペイロード 8 バイトを読み取って音色合成（倍音・ADSR 調整 → 波形生成）に渡す。part\_id は 1 対 1 接続のため検証用途で参照する。

### 3.3.3.6 命名規則・ログ書式 (EMA 準拠)

- ・ Config 型: {Module}Config (例: ImuConfig)
- ・ Config インスタンス: {MODULE}\_CONFIG (例: IMU\_CONFIG)
- ・ Data 型: {Module}Data (例: ImuData)
- ・ ログ: [ModuleName] msg 形式 (例: [OrcNet] reconnecting (attempt 3))
- ・ デバッグログは #ifdef DEBUG で切替え可能

## 3.4 処理フローの設計

### 3.4.1 共通: 3 フェーズ実行モデル

全ノード共通で、loop() は「入力 → ロジック applyPattern() → 出力」の 3 フェーズで実行する。

```

1 void loop() {
2     // 1. 入力フェーズ: センサー / UDP 受信 -> SystemData
3     for (int i = 0; i < INPUT_COUNT; i++) {
4         if (inputModules[i]->enabled)
5             inputModules[i]->updateInput(systemData);
6     }
7     // 2. ロジックフェーズ: SystemData を読み書き
8     applyPattern(systemData);
9     // 3. 出力フェーズ: SystemData -> ハードウェア / UDP 送信
10    for (int i = 0; i < OUTPUT_COUNT; i++) {
11        if (outputModules[i]->enabled)

```

```

12         outputModules[i]->updateOutput(systemData);
13     }
14 }

```

**Listing 3.1 全ノード共通の loop()**

入力モジュールは SystemData を書き込み、出力モジュールは読み込む。クロス参照は禁止。OrcNetModule は両配列に登録し、「入力フェーズで先に受信、出力フェーズの最後で送信」を配列順で制御する。

## 3.4.2 指揮者ノード (node\_01)

### 3.4.2.1 モジュール構成

- ・ 入力配列: OrcNetModule, ImuModule
- ・ 出力配列: OrcSenderModule, StatusLedModule, OrcNetModule
- ・ ロジック: applyPattern() (フィルタ・拍検出・テンポ推定・状態遷移)

### 3.4.2.2 ProjectConfig (抜粋)

```

1 // 共有バスピン
2 constexpr int I2C_SDA_PIN = 18;
3 constexpr int I2C_SCL_PIN = 19;
4
5 const ImuConfig IMU_CONFIG = {
6     .address          = 0x6A,      // LSM6DSOX
7     .sampleIntervalMs = 5,        // 200 Hz
8     .accelRangeG      = 4,
9     .gyroRangeDps     = 2000,
10 };
11 const OrcNetConfig ORC_NET_CONFIG = {
12     .ssid              = "OrchestraAP",
13     .pass              = "orchestra2026",
14     .multicastIp       = IPAddress(239, 0, 0, 1), // CTRL/BEAT 配信用
15     .udpPort           = 5001,
16 };
17 const OrcSenderConfig ORC_SENDER_CONFIG = {
18     .ctrlIntervalMs    = 50,      // 20 Hz
19     .beatRedundancy    = 1,      // 同一 BEAT を何発連送するか
20 };
21 const StatusLedConfig STATUS_LED_CONFIG = {
22     .pin               = LED_BUILTIN,

```

```

23     .blinkIntervalMs = 500,
24 };
25
26 struct LogicParams {
27     float    lpfAlpha          = 0.10f;
28     float    beatAccelThresholdG = 1.80f;
29     uint32_t beatRefractoryMs   = 250;
30     float    bpmEmaAlpha       = 0.30f;
31     float    bpmMin            = 40.0f;
32     float    bpmMax            = 240.0f;
33     uint32_t calibrationMs      = 2000;
34     uint32_t reinitRetryMs      = 5000;
35 };
36 constexpr LogicParams LOGIC_PARAMS = {};

```

Listing 3.2 node\_01/include/ProjectConfig.h (抜粋)

### 3.4.2.3 SystemData (抜粋)

ImuData・OrcNetData・OrcSenderData・StatusLedDataに加え、applyPattern() の中間状態として BeatLogicData・TempoLogicData・CalibrationData・ConductorStateData を集約する。

```

1  enum class ConductorState : uint8_t {
2      Idle = 0, Calibrating = 1, Conducting = 2, Fallback = 3,
3  };
4  struct BeatLogicData      { bool event=false; uint16_t beatNo=0; uint32_t
    lastBeatMs=0; };
5  struct TempoLogicData     { float bpm=0.0f; uint32_t nextBeatPredictedMs=0;
    uint8_t velocity=64; };
6  struct CalibrationData    { bool done=false; uint32_t startMs=0; float
    gravityOffset[3]={0,0,0}; };
7  struct ConductorStateData { ConductorState state = ConductorState::Idle; };
8
9  struct SystemData {
10     ImuData      imu;
11     OrcNetData   orcNet;
12     OrcSenderData sender;
13     StatusLedData led;
14     BeatLogicData beat;
15     TempoLogicData tempo;
16     CalibrationData calibration;
17     ConductorStateData conductor;
18 };

```

Listing 3.3 node\_01/include/SystemData.h (抜粋)

### 3.4.2.4 applyPattern() の処理フロー

毎周期、以下の順で処理する。

1. **IIR LPF + ノルム計算**: `data.imu.acc` を一次ローパス ( $\alpha = 0.10$ ) で平滑化し、`data.imu.accNorm` を計算。
2. **拍検出**: `state == Conducting` のときのみ実行。ノルムが閾値  $1.80g$  を超え、前回検出から不応期  $250\text{ ms}$  以上経過していれば `data.beat.event = true` とし `beatNo` を加算。
3. **テンポ推定 (EMA)**: 拍間隔  $\rightarrow$  瞬時 BPM  $\rightarrow \alpha = 0.30$  の指数移動平均で `data.tempo.bpm` を更新。[`bpmMin`, `bpmMax`] にクランプし `nextBeatPredictedMs` を計算。
4. **状態遷移**: `Idle`  $\rightarrow$  `Calibrating` (WiFi 接続完了)  $\rightarrow$  `Conducting` (2 秒経過)  $\rightarrow$  `Fallback` (IMU/WiFi エラー)  $\rightarrow$  `Conducting` (復旧) の判定。
5. **init 失敗モジュールの再試行**: `enabled == false` かつ 5 秒経過で `init()` を再呼び出し。

**拍検出アルゴリズム**: 候補 (ノルムピーク閾値/鉛直成分ゼロクロス/角速度ピーク) のうち、装着方向に依存せず実装が容易な **ノルムピーク閾値** を初期採用する。強い振りでないと検出しないという欠点があるため、§3.5 で精度 (MOP-3) を検証し、必要なら鉛直成分ゼロクロス方式に拡張する。

### 3.4.2.5 OrcSenderModule の出力フェーズ

```
1 void OrcSenderModule::updateOutput(SystemData& data) {
2     // BEAT: イベント駆動
3     if (data.beat.event) {
4         BeatPacket p;
5         p.header.type = TYPE_BEAT;
6         p.header.seq = ++data.sender.beatSeq;
7         p.beatNo = data.beat.beatNo;
8         for (int i = 0; i < _config.beatRedundancy; i++) {
9             _net->enqueueBeat(p);
10        }
11    }
12    // CTRL: 周期駆動 (20 Hz)
13    if (_ctrlTimer.getNowTime() >= _config.ctrlIntervalMs) {
14        _ctrlTimer.setTime();
15        CtrlPacket p;
```



```

16     p.header.type = TYPE_CTRL;
17     p.header.seq  = ++data.sender.ctrlSeq;
18     p.bpmQ8       = static_cast<uint16_t>(data.tempo.bpm * 8);
19     p.velocity    = data.tempo.velocity;
20     p.state       = static_cast<uint8_t>(data.conductor.state);
21     _net->enqueueCtrl(p);
22 }
23 }

```

Listing 3.4 OrcSenderModule.cpp の updateOutput() (要点)

### 3.4.3 楽器ノード (node\_02-05)

4 台は 同一コードで動かし、差分は ProjectConfig.h と score\_data.h (および score\_data.cpp) にのみ閉じ込める。

#### 3.4.3.1 モジュール構成

- ・ 入力配列: OrcNetModule, OrcReceiverModule
- ・ 出力配列: NoteSenderModule, StatusLedModule, OrcNetModule
- ・ ロジック: applyPattern() (楽譜進行・SelfRun 判定・発音フラグ)

#### 3.4.3.2 ProjectConfig (抜粋)

```

1  const OrcNetConfig ORC_NET_CONFIG = {
2      .ssid          = "OrchestraAP",
3      .pass          = "orchestra2026",
4      .multicastIp   = IPAddress(239, 0, 0, 1), // 指揮者と同じグループ
5      .udpPort       = 5001,
6  };
7  const OrcReceiverConfig ORC_RECEIVER_CONFIG = {
8      .partId        = 0x02, // node_02 = 金管 1
9      .startBeatNo   = 0,    // 輪唱の入り (パート毎に差分)
10     .beatTimeoutMs  = 1500, // SelfRun 発動閾値
11 };
12 const NoteSenderConfig NOTE_SENDER_CONFIG = {
13     .baudRate = 115200, // USB CDC (自パート Mac のシリアルポートへ書
        込み)
14     .partId   = 0x02, // 識別用。他ノードは 0x03/0x04/0x05 に書換え
15 };
16 const StatusLedConfig STATUS_LED_CONFIG = {
17     .pin          = LED_BUILTIN,
18     .blinkIntervalMs = 500,

```

```
19 };
```

Listing 3.5 node\_02/include/ProjectConfig.h (抜粋)

### 3.4.3.3 楽譜データ形式

楽譜は C 配列としてヘッダに埋め込み、SD カード等を使わない。イベント単位の構造体を時系列順に並べる。

```
1 struct ScoreEvent {
2     uint16_t beatAt;           // この拍 (startBeatNo 相対) で発動
3     uint8_t  noteNumber;       // MIDI ノート番号. 0 なら休符
4     uint8_t  velocity;         // 個別 velocity (CTRL と乗算合成)
5     uint16_t durationQ8;       // 拍の 1/256 単位 (256 = 1 拍)
6     uint8_t  flags;           // bit0=NoteOn, bit1=NoteOff, bit2=Rest
7 };
8 extern const ScoreEvent kScore[];
9 extern const uint16_t kScoreLength;
```

Listing 3.6 node\_02/include/score\_data.h

NoteOff の管理は durationQ8 ぶん経過時点で applyPattern() が pendingOff を立てる。NoteOff を個別イベントとして並べる必要は原則ない (flags bit1 は拡張用)。

### 3.4.3.4 applyPattern() の処理フロー

1. **演奏状態遷移:** Idle → WaitStart (WiFi 接続完了) → Playing (lastBeatNo ≥ startBeatNo) → SelfRun (beatTimeoutMs 超過) → Playing (実 BEAT 復帰)。
2. **仮想 BEAT 内挿:** SelfRun 中は最後の currentBpm から virtualBeatNo を進める。
3. **楽譜進行:** effectiveBeatNo - startBeatNo を相対拍として、kScore[currentEventIndex].beatAt 以下のすべてのイベントを順次発火。NoteOn なら pendingOn = true と noteOffAtMs を計算。
4. **velocity 合成:** 楽譜の velocity と CTRL の currentVelocity を  $\frac{\text{score} \times \text{ctrl}}{127}$  で乗算合成。ストレッチ未実装時は CTRL velocity が固定 64。
5. **NoteOff 判定:** noteIsSounding && now ≥ noteOffAtMs で pendingOff = true。

NOTE 送出は出力フェーズで NoteSenderModule が担当する。data.noteOut.pendingOn/Off を見て 20 バイトの NotePacket を組み立て、Serial.write で **自パート Mac の**シリアルポートへ一括書込みする (Mac 側は magic 0x4F52 を探してフレーム同期し、type = 3 のとき NOTE ペイロードを読み取る)。

### 3.4.3.5 ノード間差分

4 台の楽器は ProjectConfig.h と score\_data.h 以外は同一コード (表 3.5).

表 3.5: 楽器ノード間差分

| ノード     | partId      | startBeatNo | score_data.h            |
|---------|-------------|-------------|-------------------------|
| node_02 | 0x02 (金管 1) | 0           | 金管 1 の楽譜 (先頭から)         |
| node_03 | 0x03 (金管 2) | 4           | 金管 2 (輪唱で 4 拍遅れて<br>入る) |
| node_04 | 0x04 (金管 3) | 8           | 金管 3 (輪唱で 8 拍遅れて<br>入る) |
| node_05 | 0x05 (ドラム)  | 0           | ドラム (先頭からリズム<br>伴奏)     |

楽譜の beatAt は **開始ビート相対** (各パート楽譜は 0 始まり) で記述し, 楽譜データの使い回しを効かせる.

## 3.5 テストの設計

§2.1 で定義した MOP を実機で測るための試験計画と, 要求 → 検証の対応関係を示す.

### 3.5.1 試験戦略 (単体 → 結合 → システム)

PlatformIO の test/ を使い, 共通層と applyPattern() を中心に単体テストを整備する. EMA に従い判断ロジックは IModule 派生でなく applyPattern() 関数として書くため, テストでは SystemData をテスト側で組み立てて applyPattern() を呼ぶだけでロジック単体を検証できる.

表 3.6: 単体テスト対象

| 対象          | 確認内容               | 手段              |
|-------------|--------------------|-----------------|
| ModuleTimer | setTime() 後の経過時間判定 | millis() モック差替え |

| 対象                     | 確認内容                             | 手段       |
|------------------------|----------------------------------|----------|
| OrcProtocol シリアルライズ    | CTRL/BEAT/NOTE のラウンドトリップ         | バイト列比較   |
| applyPattern() 拍検出     | 記録 IMU 時系列で拍位置一致                 | ゴールデンテスト |
| applyPattern() テンポ推定   | 一定間隔の拍で BPM 収束                   | 単調列テスト   |
| applyPattern() 楽譜進行    | lastBeatNo 増加で正しい順序で pendingOn   | 状態機械テスト  |
| applyPattern() SelfRun | 時間経過で state==SelfRun, 仮想 BEAT 進行 | 状態機械テスト  |

結合テストは Tier 単位で段階的に積み上げる。

表 3.7: 結合テスト Tier

| Tier | 構成                      | 確認事項                                    | 機材                        |
|------|-------------------------|-----------------------------------------|---------------------------|
| T1   | node_01 単体              | シリアルに beat_no/bpm が出る。<br>手で振って拍が出る     | PC + USB                  |
| T2   | node_01<br>+ node_02    | node_02 が BEAT 受信 → NOTE 出力             | 受信スクリプト                   |
| T3   | node_01<br>+ node_02-05 | 4 台がほぼ同時に NOTE. 同期誤差<br>≤ 30 ms (MOP-1) | USB ハブ + ログ集約             |
| T4   | 全ノード + PC<br>Processing | 実音再生. 指揮速度を変えると<br>BPM が追従              | PC + スピーカ<br>+ Processing |

### 3.5.2 試験項目 (TPM)

各 MOP に対する試験項目を表 3.8 に示す。

表 3.8: 試験項目 (TPM)

| ID   | 項目                     | 目標                     | 手順                                                                                     | 合否基準                  | 時期       |
|------|------------------------|------------------------|----------------------------------------------------------------------------------------|-----------------------|----------|
| TP-1 | 拍 検 出 精<br>度           | MOP-3: $\geq$<br>90%   | 80/120/160 BPM のメ<br>トロノームに合わせて<br>指揮し, 30 秒の検出 vs<br>実拍数                              | $\geq 90\%$           | W6 (M3)  |
| TP-2 | 通信遅延                   | MOP-2: $\leq$<br>10 ms | node_01 の 平均 $\leq 10$<br>timestamp_ms と ms<br>node_02 受 信 時 刻<br>の 差 を 100 サンプル<br>平均 |                       | W7 (M4)  |
| TP-3 | 同期誤差                   | MOP-1: $\leq$<br>30 ms | 4 ノードのシリアル口<br>グを PC で時刻同期して<br>BEAT 受信時刻を比較                                           | 最 大 差 $\leq$<br>30 ms | W9 (M5)  |
| TP-4 | テ ン ポ 追<br>従           | MOP-4: $\leq$<br>2 拍   | 80 $\rightarrow$ 120 BPM 切替時に<br>楽器側 BPM が 120 に入<br>るまでの拍数                            | $\leq 2$ 拍            | W9 (M5)  |
| TP-5 | パ ケ ロ ス<br>耐性          | MOP-5                  | ルータで擬似 5% パケロ<br>スを設定, 演奏破綻なき<br>こと                                                    | 聴 感 破 綻<br>なし         | W11 (M7) |
| TP-6 | CPU 負荷                 | MOP-6: $\leq$<br>2 ms  | micros() で入力フェー<br>ズ所要時間を 10 分ログ<br>取得                                                 | 最 大 $\leq 2$<br>ms    | W11 (M7) |
| TP-7 | 起動時間                   | MOP-7: $\leq$<br>5 s   | 電源投入 $\rightarrow$ 最初の CTRL<br>送出までを 5 回平均                                             | $\leq 5$ s            | W11 (M7) |
| TP-8 | 強 弱 制 御<br>(ストレッ<br>チ) | MOP-8                  | 小・中・大の振り幅で<br>velocity 出力値を記録                                                          | 3 段階分離                | W10 (M6) |

### 3.5.3 要求 → 検証の対応

表 3.9: 要求と検証の対応表

| 要求 ID        | 要求内容                  | 検証方法                  | 関連 MOP       |
|--------------|-----------------------|-----------------------|--------------|
| R-1          | 5 台同期演奏               | T4 で聴感確認 + MOP-1/2 実測 | MOP-1, MOP-2 |
| R-2          | 輪唱曲（主旋律 3 + リズム 1 以上） | 4 パート楽譜を用意し T4 で演奏    | —            |
| R-3          | 可変テンポ追従               | 指揮 BPM 階段変化で実測        | MOP-4        |
| R-4 (S)      | 強弱制御                  | 指揮振幅変化で               | MOP-8        |
| R-Internal-1 | 通信の信頼性                | MOP-5（5% パケロスで継続）     | MOP-5        |
| R-Internal-2 | CPU リソース              | MOP-6（入力 $\leq 2$ ms） | MOP-6        |
| R-Internal-3 | 起動時間                  | MOP-7（5 s 以内）         | MOP-7        |